Specify the operation, such as key-based lookup, FIFO removal or index access, so there is one defensible correct answer.

IMPROVE: Explain one correction to your own reasoning.

DO NOW — RECALL

C# Login Project · Start from Scratch

Name: _____ Class: _____ Date: _____

1. A rising-edge-triggered D-type flip-flop has Q = 0 and D = 1. What happens to Q when the next rising edge occurs?

2. Why do we group neighbouring 1s together on a Karnaugh map?

3. Push A, then B, then C onto an empty stack. Pop once and then push D. Which item is at the top, and what is the remaining stack from top to bottom?

4. Using 8-bit two’s complement, write −13 and explain why simply putting 1 before the 7-bit magnitude is not correct.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Q becomes 1 at the rising edge because the flip-flop samples D. It then holds that value until the next triggering edge.

2. Model answer

We group neighbouring 1s to simplify the Boolean expression. Variables that change within the group can be removed.

3. Model answer

D is at the top. From top to bottom the stack is D, B, A. C was removed by the pop.

4. Model answer

13 = 00001101_2. Invert to 11110010_2, then add 1: 11110011_2. Prefixing the magnitude gives sign-and-magnitude (10001101_2), not two’s complement.

By the end, you can…

• Write a C# login program from a blank editor.

• Use input, selection and a loop to validate a demonstration account.

• Write four synoptic MCQs about arrays, records, lists and tuples.

Vocabulary — know these terms

sequence — Statements executed in order.

selection — Branching with if/else.

iteration — Repeating statements with a loop.

validation — Checking input meets defined rules.

array — Indexed fixed-size collection, possibly multidimensional.

record — Named fields grouped as a single item.

list — Collection that can grow or shrink.

tuple — Ordered fixed group of values.

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Build the condition

CODE:

```
if (username == "student" && password == "demo123")
{
Console.WriteLine("Access granted");
}
else
{
Console.WriteLine("Access denied");
}
```

IMPROVE: Explain one correction to your own reasoning.

Guided 2 — original CHECK

Explain your solution first

How could you allow at most three attempts?

Handle repeated attempts

CODE:

```
for (int attempt = 1; attempt <= 3; attempt++)
{
// Read inputs, check both values, break on success.
}
```

IMPROVE: Explain one correction to your own reasoning.

Guided 3 — original CHECK

Explain your solution first

A student writes: 'Which structure is best?' Why is this too vague?

MCQ quality check

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A login should allow access when the username is correct even if the password is wrong.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

NEW LEARNING

Read, annotate and explain

Build the first login

Classroom demonstration only. Never type, collect or store real passwords.

What must a login program do?

Read two inputs, decide whether both match the demonstration account, and show a clear result. Design the sequence before typing code.

Why start with sequence?

Read username; read password; display what was entered only during debugging. In the final version never echo a password.

Introduce selection

Use AND (&&) to require BOTH username and password to match. One wrong value must mean access denied.

Make a second attempt possible

A loop can repeat after invalid credentials. Add a maximum number of attempts to avoid an endless loop.

Data-structure revision

Write one accurate four-option MCQ each on arrays (up to 3D), records, lists and tuples. Add answers and distractor explanations.

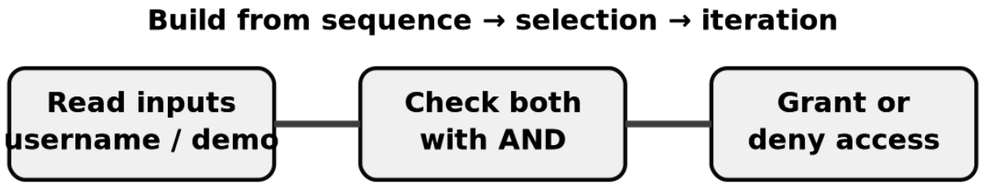
Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Build the first login

Classroom demonstration only. Never type, collect or store real passwords.



Input username and demo password, compare, then grant or deny access

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

CODING 3 · ITERATION
Extend the program to allow at most three attempts. Show 'Locked for this session' after three unsuccessful attempts.

Question 4

SYNOPTIC MCQ BANK
Write FOUR original four-option MCQs: one on arrays up to 3D, one on records, one on lists and one on tuples. Mark the correct letter, explain it and justify each distractor.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

CODING 1 · SEQUENCE
From a blank C# console program, request a username and a demonstration password. Use two string variables. Do not copy in a finished solution.

Question 2

CODING 2 · SELECTION
Add an if/else. Grant access only for username 'student' AND demo password 'demo123'. Test correct name/wrong password and vice versa.

I DO — WORKED EXAMPLE

Follow the worked method

Model the first three lines — not the finished program

Create a console program

```
using System;

Console.Write("Username: ");
string username = Console.ReadLine() ?? "";
```

Add the second input

```
Console.Write("Demo password: ");
string password = Console.ReadLine() ?? "";
```

Plan, then add selection

```
// Write an if statement:
// username must equal "student"
// AND password must equal "demo123"
// Print "Access granted" or "Access denied".
```

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Build the condition

What Boolean operator makes BOTH checks compulsory?

Method / working:

2. Handle repeated attempts

How could you allow at most three attempts?

Method / working:

3. MCQ quality check

A student writes: 'Which structure is best?' Why is this too vague?

Method / working:

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Build the condition What Boolean operator makes BOTH checks compulsory? Explain your solution first

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 2: Handle repeated attempts How could you allow at most three attempts?

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 3: MCQ quality check A student writes: 'Which structure is best?' Why is this too vague?

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.